# Self-defined Topic:
# A critical study on Arm SME ISA and RISC-V processors

Pei-Yu Lin

## Questions

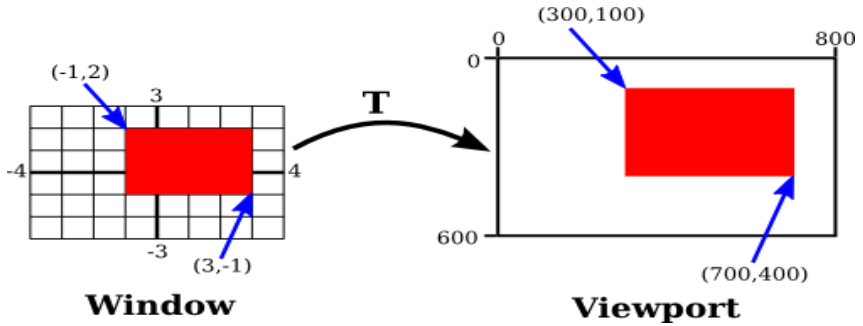
1. How does Arm make the Matrix ISA scalable across different vector implementations?
2. How do instructions interact between vector and matrix registers?
3. How to make an Arm SME-like RISC-V Vector-Matrix Extensions?

Chair of Integrated Digital Systems and Circuit Design
RWTH AACHEN UNIVERSITY

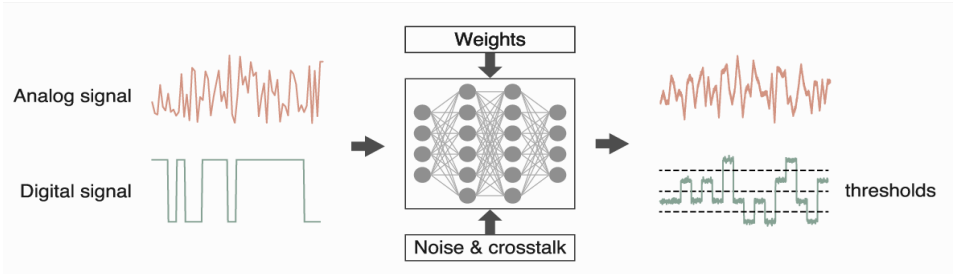
# Matrix Multiplication (MatMul)

## Which fields need Matrix Multiplication

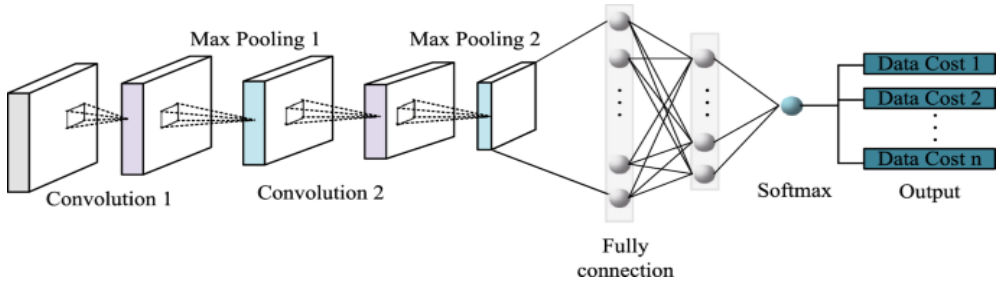
- Computer Graphics

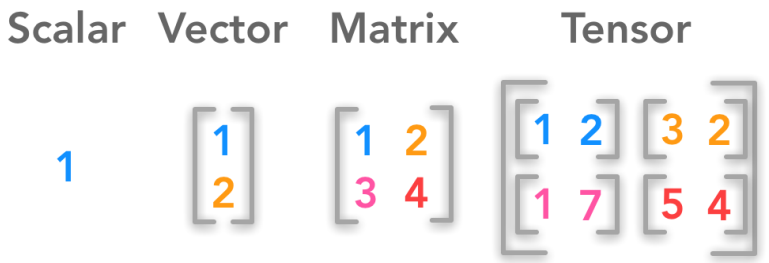



- Digital Signal Processing



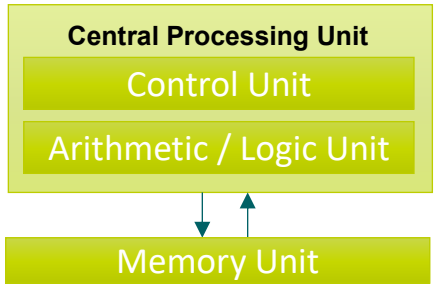

- Machine Learning




- Because of DNN, Matrix-based workload ↑

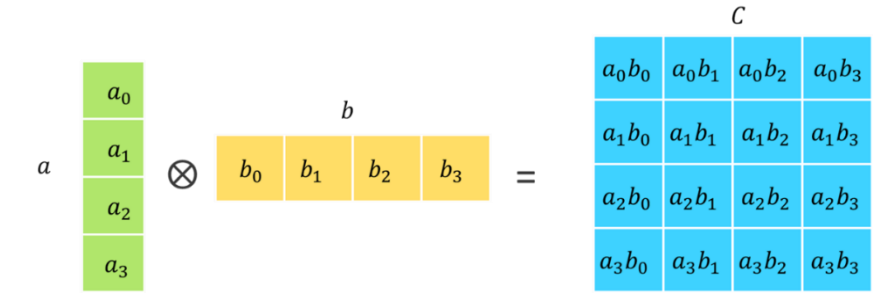
Vector Architecture → Matrix Architecture



# How a matrix multiplication can be performed using the outer product operation?

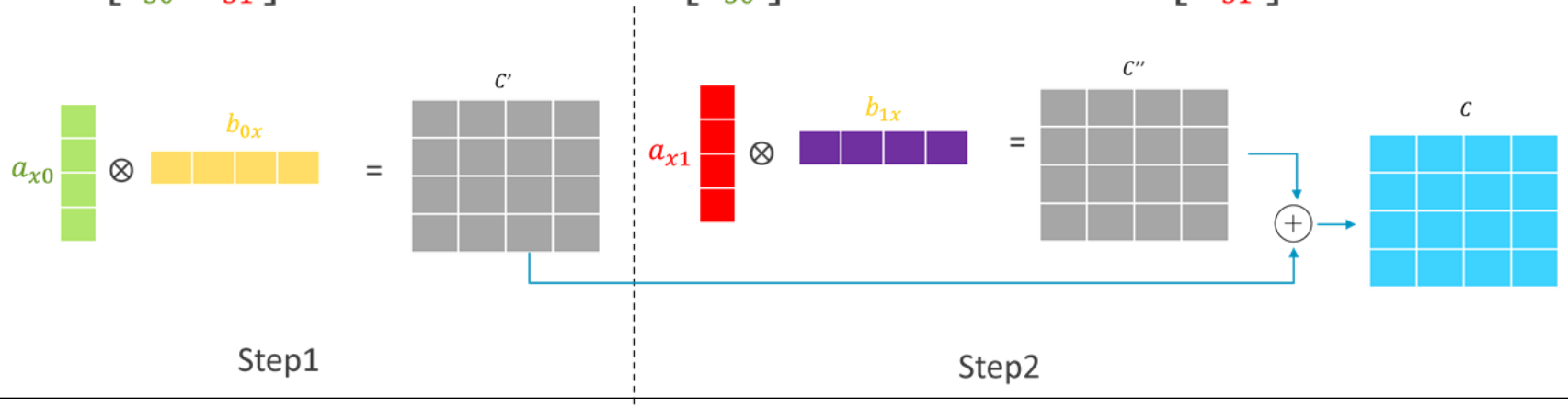
Calculating the outer product of two vectors a and b gives a result matrix C containing the outer product:

$$C = a \otimes b = ab^T = \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} [b_0 \quad b_1 \quad b_2 \quad b_3] = \begin{bmatrix} a_0b_0 & a_0b_1 & a_0b_2 & a_0b_3 \\ a_1b_0 & a_1b_1 & a_1b_2 & a_1b_3 \\ a_2b_0 & a_21 & a_2b_2 & a_2b_3 \\ a_3b_0 & a_31 & a_3b_2 & a_3b_3 \end{bmatrix}$$




Now consider multiplying two matrices, a and b:

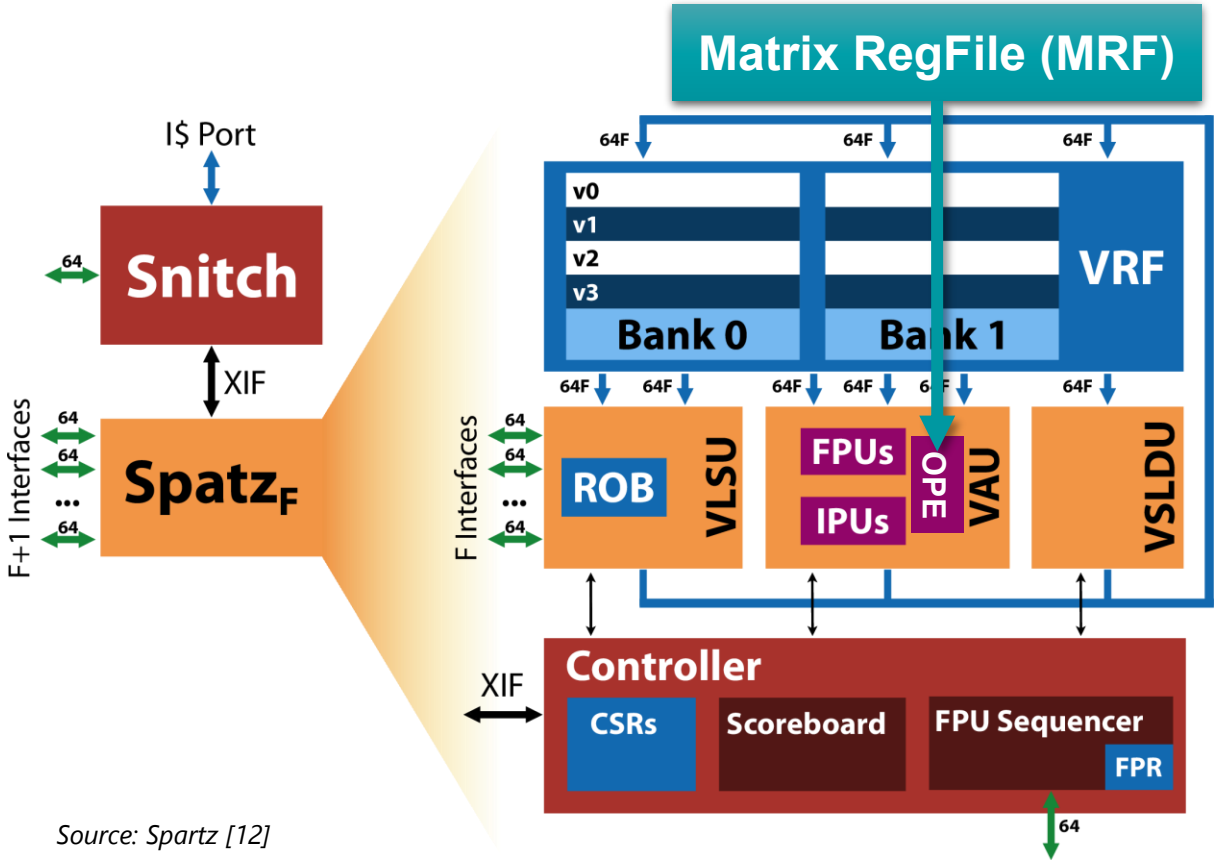
$$C = a * b = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \\ a_{20} & a_{21} \\ a_{30} & a_{31} \end{bmatrix} \begin{bmatrix} b_{00} & b_{01} & b_{02} & b_{03} \\ b_{10} & b_{11} & b_{12} & b_{13} \end{bmatrix} = \begin{bmatrix} a_{00} \\ a_{10} \\ a_{20} \\ a_{30} \end{bmatrix} [b_{00} \quad b_{01} \quad b_{02} \quad b_{03}] + \begin{bmatrix} a_{01} \\ a_{11} \\ a_{21} \\ a_{31} \end{bmatrix} [b_{10} \quad b_{11} \quad b_{12} \quad b_{13}]$$




*Source: Arm Blog [8]*

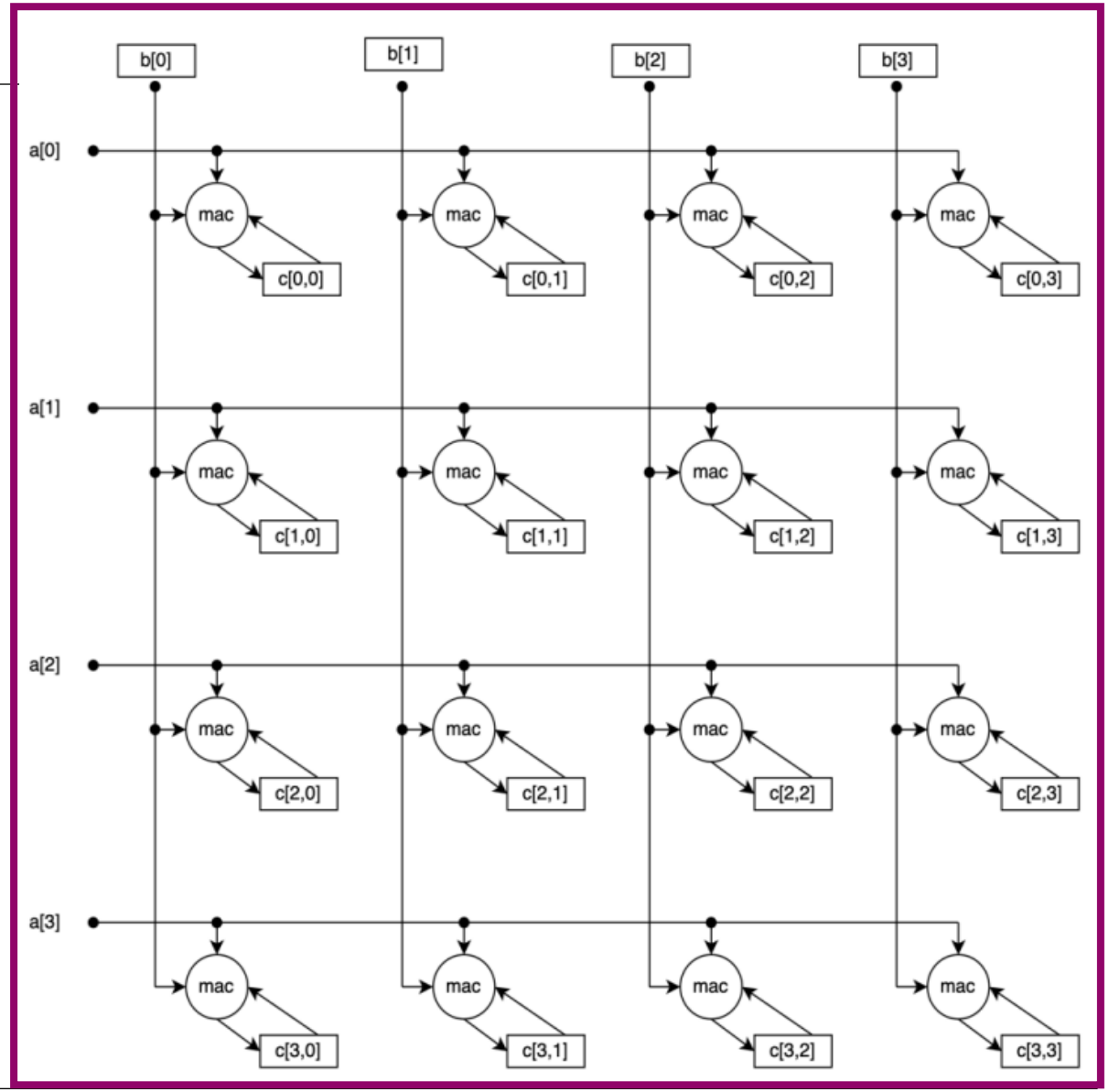
# Integrate OPE & MRF into Spatz

## RISC-V Vector Processor: Spatz




*Source: Spartz [12]*

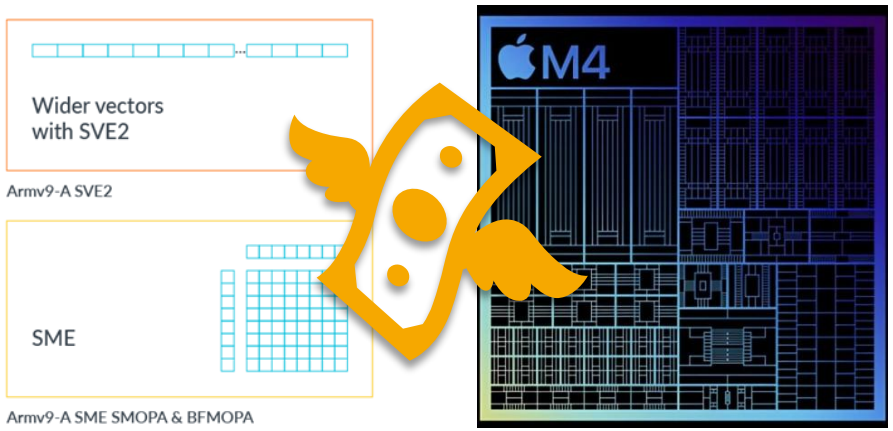
**Outer Product Engine is a Distributed Architecture**



# Instructions for Outer Product Engine in RISC-V Vector Processor

Arm released SME in 2021

Apple M4 applies Armv9 architecture

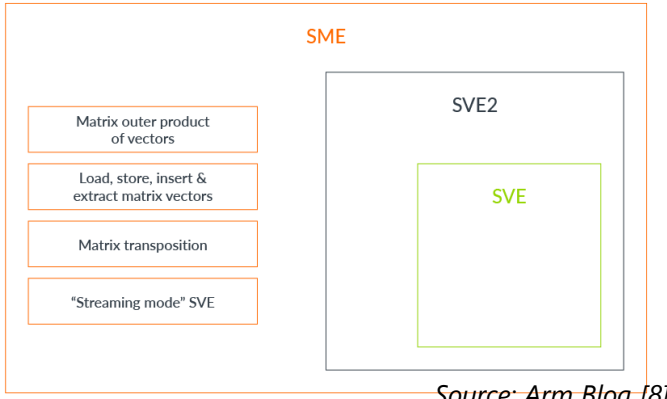



Custom extension

ARM - like RISC-V

- Proposed Vector Matrix Extension
- Open-source
- (-) No standardized VME

## What is ARM Scalable Matrix Extension (SME)?




Includes a subset of the SVE2 and adds new SME-specific instructions.

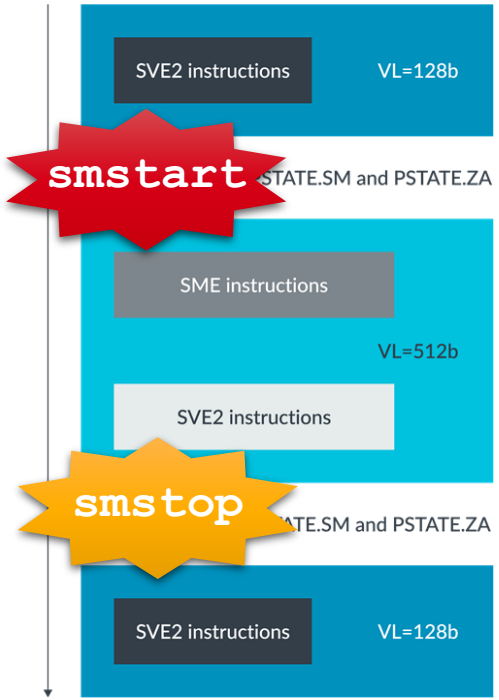
*Source: Arm Blog [8]*

# Arm SME's Key Feature

- general-purpose
- complex data structures
- complex predication (branch)

- high-throughput
- large datasets
- Simple control flow

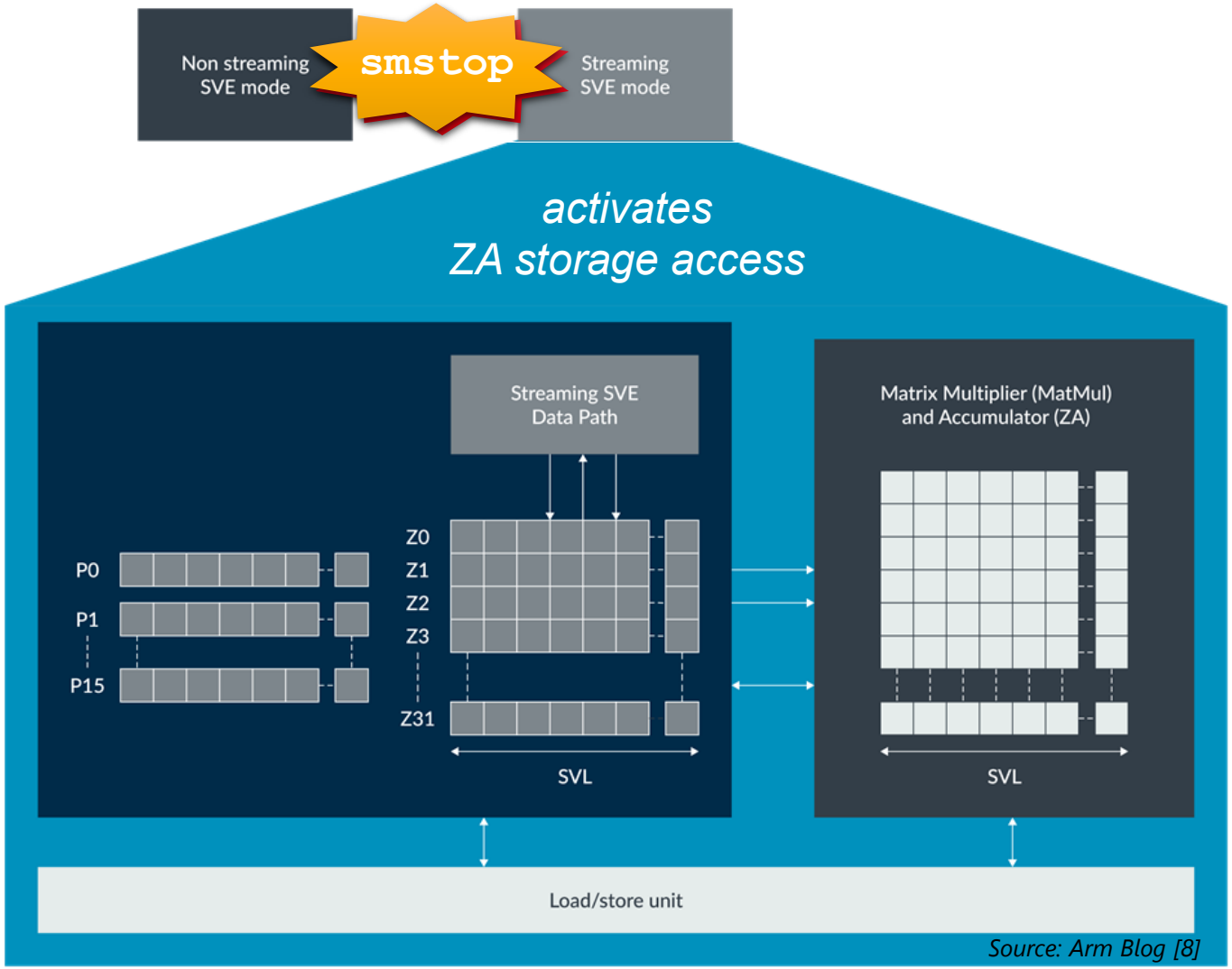
## Streaming SVE switching mode




$$SVL \geq VL$$

SVL (streaming vector length) can be 128-bit, 256-bit, 512-bit, 1048-bit, and 2056-bit

- (+) longer SVL better ultilizes memory bandwidth
- (+) shorter VL reduces energy and latency during accessing
- integrates both matrix and vector operations within same HW
  e.g. 2D Convolution (SVL) → 1D RELU (VL)




*Source: Arm Blog [8]*

# Scalable Vector Extension (SVE)

e.g. matmul with 11 elements in VLEN=4 elems register

**NEON**

```
for (int i_idx=0; i_idx<n; i_idx+=4) {
   for_loop_j
      for_loop_k      iter=0, 1
}

for (int i_idx=(n-4); i_idx<n; i_idx+=3) {
    for_loop_j'
       for_loop_k'    iter=2 (cleanup code)
}
```

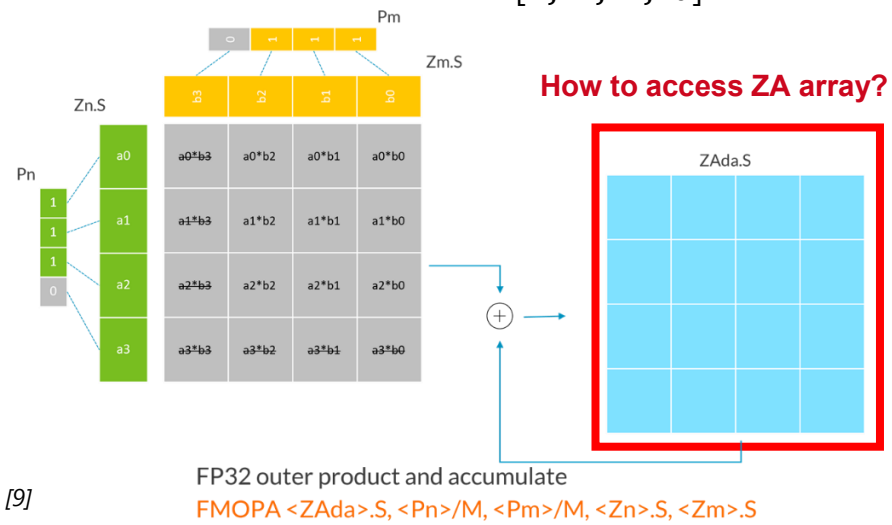
8 if VL=128-bit

**SVE/SVE2**

```
for (int i_idx=0; i_idx<n; i_idx+=svcntw()) {
// calculate predicate for this i_idx
   svbool_t pred = svwhilelt_b32_u32(i_idx, n);
   for_loop_j
      for_loop_k
```

P0=[1, 1, 1, 1]
P1=[1, 1, 1, 1]
P2=[1, 1, 1, 0]

**SME/SME2**



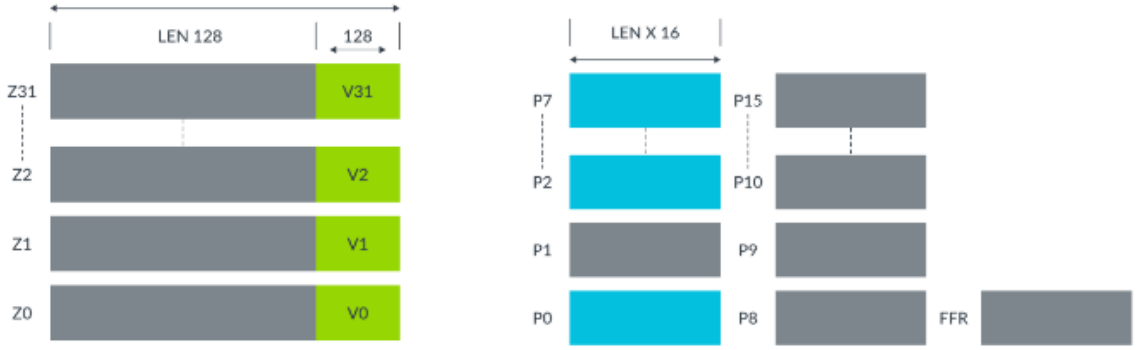

*Source: Arm Blog [9]*

## Vector length agnostic (VLA)

Use a counter to record # of elements (32-bit) in one vector; for-loop stride = `cntw()`

## Predicate Register

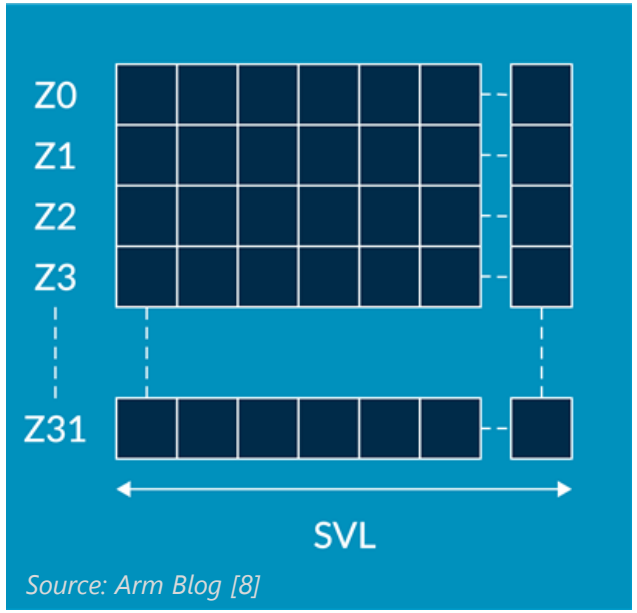
to mask partial vector operation at matrix edges
(+) no need explicit cleanup code when handling a loop tail or pivoting in matrix factorization



# Access approach 1:  ZA array vector

## ZA array

2D square byte array with dimesion (SVL in bytes) x (SVL in bytes)



*Source: Arm Blog [8]*

- The ZA array can be accessed as SVL-bit vectors that contain 8-bit, 16-bit, 32-bit, 64-bit, or 128-bit elements:
  - `ZA.B[N], ZA.H[N], ZA.S[N], ZA.D[N], ZA.Q[N]`
- N = # of ZA array vectors = # of bytes in SVL
  - E.g. if SVL is 256-bit, the number of ZA array vectors is 32.
- New instr for load and store ZA array vectors ⟷ memory:
  - `LDR ZA[<Wv>, <imm>], [<Xn|SP>{, #<imm>, MUL VL}]`
  - `STR ZA[<Wv>, <imm>], [<Xn|SP>{, #<imm>, MUL VL}]`

# Access approach 2:  ZA tile

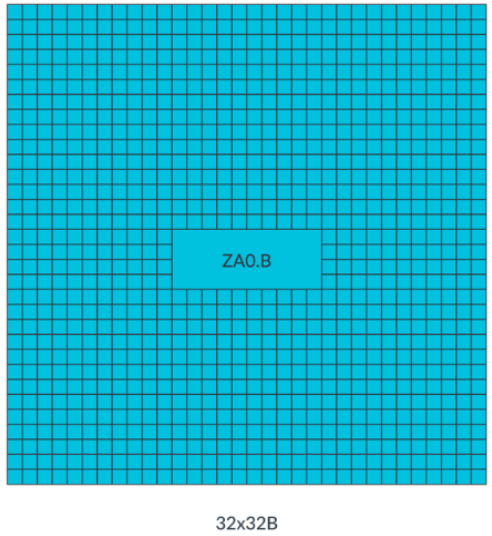
## ZA tile

- a square, 2D sub-array of elements within the ZA array.
- Use the tile name to access a whole ZA tile
- The number of tiles is determined by the element data type size.

| Element data type size | Number of tiles | Tile names |
|---|---|---|
| 8-bit | 1 | ZA0.B |
| 16-bit | 2 | ZA0.H-ZA1.H |
| 32-bit | 4 | ZA0.S-ZA3.S |
| 64-bit | 8 | ZA0.D-ZA7.D |
| 128-bit | 16 | ZA0.Q-ZA15.Q |

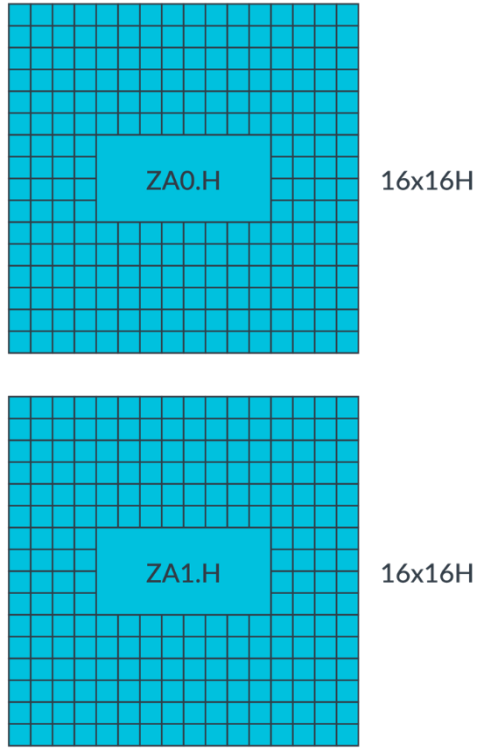
## Example: SVL = 256-bit (32-byte)

(1) the element data type size is 8-bit, ZA can be viewed as ZA0.B, and as 32 x (32 x 1-byte) vectors.




*Source: Arm Blog [8]*

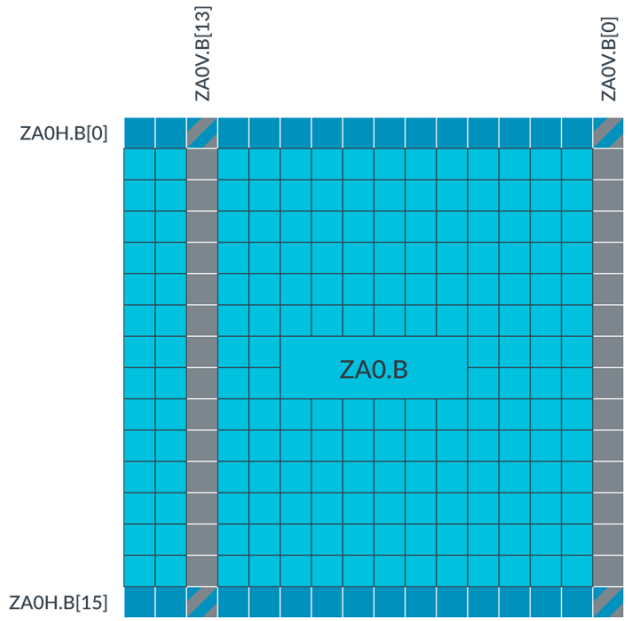
(2) the element data type size is 16-bit, ZA can be viewed as two tiles, ZA0.H and ZA1.H, and each tile can be viewed as 16x (16x 2-byte) vectors.



# Access approach 3: ZA tile slice

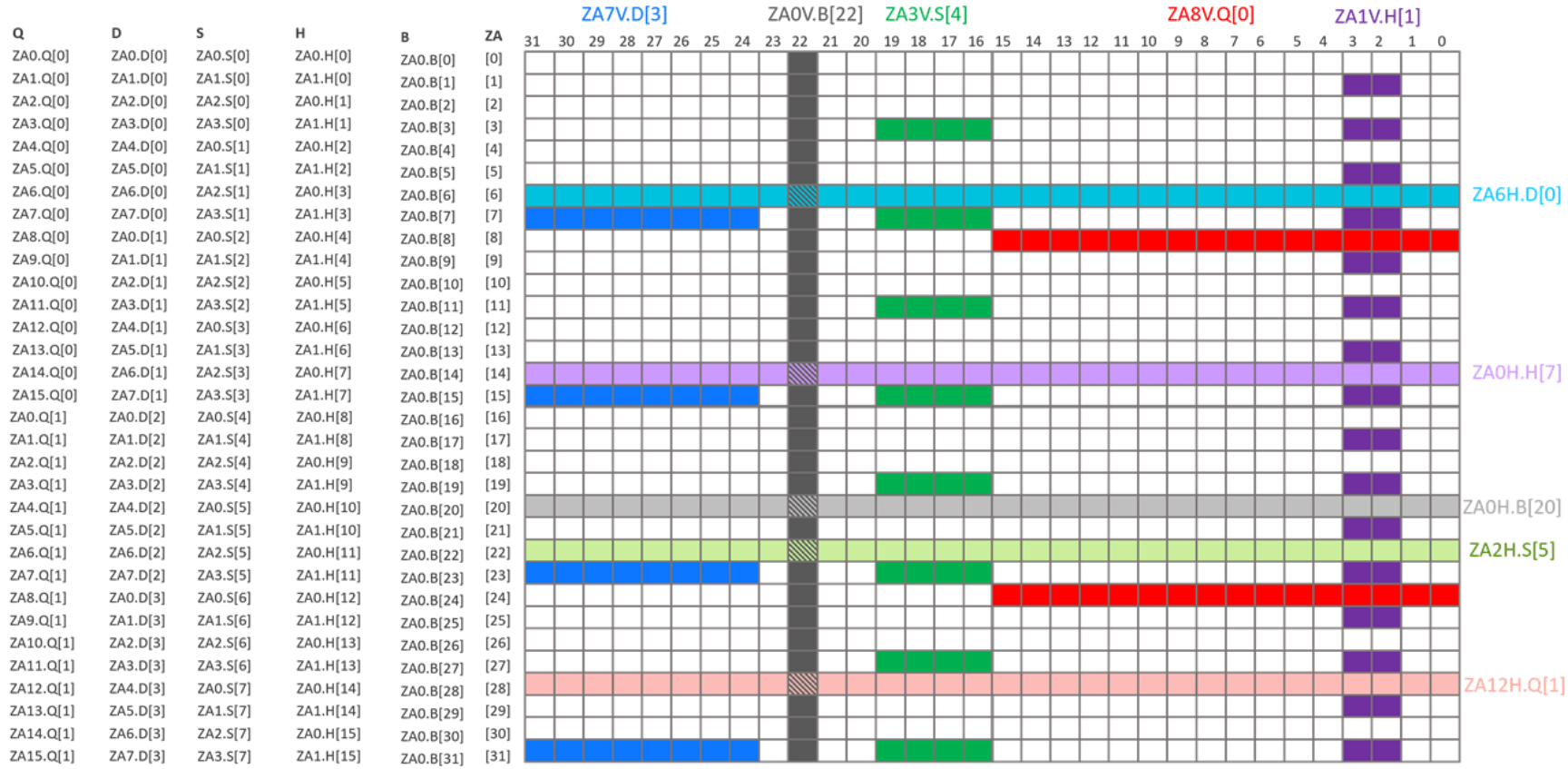
## ZA tile slice

- a 1D set of horizontally or vertically contiguous elements within a ZA tile.




Example of the combined view of different element data type sizes, and horizontal and vertical tile slices.

| Q | D | S | H |
|---|---|---|---|
| ZA0.Q[0] | ZA0.D[0] | ZA0.S[0] | ZA0.H[0] |
| ZA1.Q[0] | ZA1.D[0] | ZA1.S[0] | ZA1.H[0] |
| ZA2.Q[0] | ZA2.D[0] | ZA2.S[0] | ZA0.H[1] |
| ZA3.Q[0] | ZA3.D[0] | ZA3.S[0] | ZA1.H[1] |
| ZA4.Q[0] | ZA4.D[0] | ZA0.S[1] | ZA0.H[2] |
| ZA5.Q[0] | ZA5.D[0] | ZA1.S[1] | ZA1.H[2] |
| ZA6.Q[0] | ZA6.D[0] | ZA2.S[1] | ZA0.H[3] |
| ZA7.Q[0] | ZA7.D[0] | ZA3.S[1] | ZA1.H[3] |
| ZA8.Q[0] | ZA0.D[1] | ZA0.S[2] | ZA0.H[4] |
| ZA9.Q[0] | ZA1.D[1] | ZA1.S[2] | ZA1.H[4] |
| ZA10.Q[0] | ZA2.D[1] | ZA2.S[2] | ZA0.H[5] |
| ZA11.Q[0] | ZA3.D[1] | ZA3.S[2] | ZA1.H[5] |
| ZA12.Q[0] | ZA4.D[1] | ZA0.S[3] | ZA0.H[6] |
| ZA13.Q[0] | ZA5.D[1] | ZA1.S[3] | ZA1.H[6] |
| ZA14.Q[0] | ZA6.D[1] | ZA2.S[3] | ZA0.H[7] |
| ZA15.Q[0] | ZA7.D[1] | ZA3.S[3] | ZA1.H[7] |
| ZA0.Q[1] | ZA0.D[2] | ZA0.S[4] | ZA0.H[8] |
| ZA1.Q[1] | ZA1.D[2] | ZA1.S[4] | ZA1.H[8] |
| ZA2.Q[1] | ZA2.D[2] | ZA2.S[4] | ZA0.H[9] |
| ZA3.Q[1] | ZA3.D[2] | ZA3.S[4] | ZA1.H[9] |
| ZA4.Q[1] | ZA4.D[2] | ZA0.S[5] | ZA0.H[10] |
| ZA5.Q[1] | ZA5.D[2] | ZA1.S[5] | ZA1.H[10] |
| ZA6.Q[1] | ZA6.D[2] | ZA2.S[5] | ZA0.H[11] |
| ZA7.Q[1] | ZA7.D[2] | ZA3.S[5] | ZA1.H[11] |
| ZA8.Q[1] | ZA0.D[3] | ZA0.S[6] | ZA0.H[12] |
| ZA9.Q[1] | ZA1.D[3] | ZA1.S[6] | ZA1.H[12] |
| ZA10.Q[1] | ZA2.D[3] | ZA2.S[6] | ZA0.H[13] |
| ZA11.Q[1] | ZA3.D[3] | ZA3.S[6] | ZA1.H[13] |
| ZA12.Q[1] | ZA4.D[3] | ZA0.S[7] | ZA0.H[14] |
| ZA13.Q[1] | ZA5.D[3] | ZA1.S[7] | ZA1.H[14] |
| ZA14.Q[1] | ZA6.D[3] | ZA2.S[7] | ZA0.H[15] |
| ZA15.Q[1] | ZA7.D[3] | ZA3.S[7] | ZA1.H[15] |




*Source: Arm Blog [8]*

# Common Arm’s SME instructions

- Instructions that accumulate or subtract the outer product of two vectors into a ZA tile
- Load, store, and move instructions that transfer a vector to or from a ZA tile row or column
- Instructions that add a vector horizontally or vertically to a ZA tile
- instructions that add a multiple of the vector size in Streaming SVE mode to a scalar register

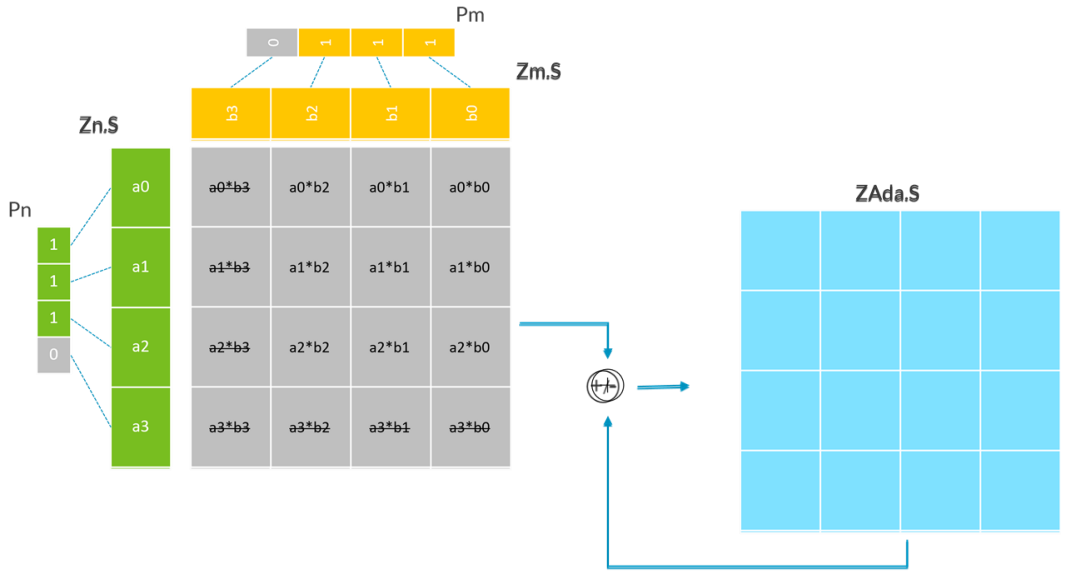
# (1) Outer product and accumulate or subtract instructions (MOPS)

Support different data type: 8-bit and 16-bit integer, FP16, BF16, FP32, and FP64 floating point

## Examples with SVL=128-bit

Case 1: FP32, FP64

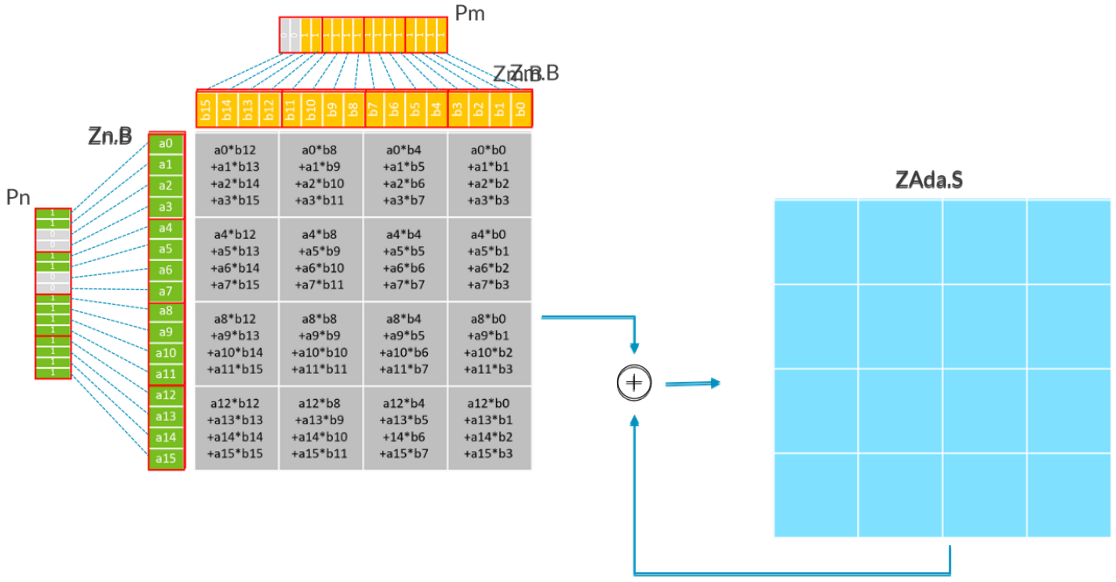
- FMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.S, <Zm>.S
- FMOPS <ZAda>.Q, <Pn>/M, <Pm>/M, <Zn>.Q, <Zm>.Q




FMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.S, <Zm>.S

Case 2: FP16, BF16, INT16, INT8, I16I64

- UMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.B, <Zm>.B
- BFMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H



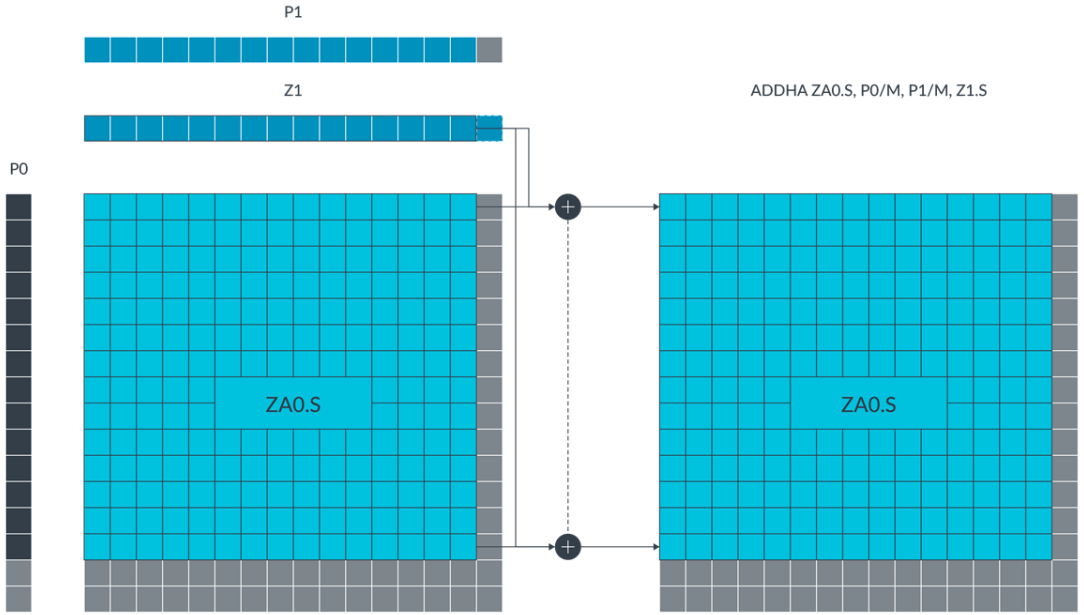

UMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.B, <Zm>.B

*Source: Arm Blog [9]*

# (2) Other instructions

## Addition of a vector to ZA rows and columns

- `ADDHA ZA0.S, P0/M, P1/M, Z1.S`
- `ADDVA ZA0.S, P0/M, P1/M, Z1.S`




*Source: Arm Blog [9]*

## Tile slice load and store instructions

- LD1B, LD1H, LD1S, LD1D, and LD1Q
  - `LD1B ZA0H.B[W0, #imm], P0/Z, [X1, X2]`
- ST1B, ST1H, ST1S, ST1D, and ST1Q
  - `ST1H ZA1V.H[W0, #imm], P2, [X1, X2, LSL #1]`

## Tile slice move instructions

- `MOV  ZA0H.B[W0, #imm],  P0/M, Z0.B`
- `MOVA  ZA0H.B[W0, #imm],  P0/M, Z0.B`

## ZA array vector load/store instructions

- `STR ZA[<Wv>, <imm>], [<Xn|SP>{, #<imm>, MUL VL}]`

## Zero ZA tile instruction

- `ZERO { <mask>}`

# Proposed RISC-V Vector-Matrix Extension (VME)

TABLE I: Instruction Mapping: Arm SME vs.Proposed RISC-V VME

| Functional Category | Arm SME | Proposed RISC-V VME | Execution Unit |
|---|---|---|---|
| Matrix Outer Product Accumulate / Subtract | FMOPA | vme.mopa.vv | Outer Product Engine |
| | FMOPS | vme.mops.vv | |
| Vector-Tile Add (Horiz. / Vert.) | ADDHA | vme.addh.va | Accumulator ALU |
| | ADDVA | vme.addv.va | |
| Tile Slice Movement (Insert / Extract) | MOVA | vme.mov.v.acc | Slice Move Unit |
| | | vme.mov.acc.v | |
| Transposed Access (Row / Column) | ZA0H.S | za0h.s | Slice Move Unit |
| | ZA0V.S | za0v.s | |
| State Initialization | ZERO | vme.tile.zero | MRF Controller |
| Context Management (Load / Store) | LDR | vme.ldr | LSU / DMA |
| | STR | vme.str | |

→ `vme.mopa.vv [acc], [vs2], [vs1]`

`vme.addh.va [acc], [vs1]`
add vector vs1 to every horizontal row or vertical column of an MRF tile.

→ `vme.mov.v.acc [acc],[vs2], [rs1]`

**Transpose-on-the-fly** : facilitate data exchange between 1D vector register and 2D array.

→ `vme.tile.zero [acc]`
clears the specified MRF tile

# Simulation Setup

## (1) Toolchain: LLVM Backend Definition

```
// --- Register Class: Matrix Register File (MRF) ---
def VME_ACC : RISCVRegisterClass<[v32f32], 512, (add
     (sequence "MRF%u", 0, 7))>;

// --- 1. Arithmetic Operations (OPE) ---
def VME_MOPA_VV : RVInstVV<0b000000, 0b111,
    OPC_Custom_3, (outs VME_ACC:$acc), (ins VME_ACC:
    $acc_wb, VR:$vs2, VR:$vs1), "vme.mopa.vv", "$acc
    , $vs2, $vs1"> { let Constraints = "$acc =
    $acc_wb"; }

def VME_ADDH_VA : RVInstV<0b000101, 0b111,
    OPC_Custom_3, (outs VME_ACC:$acc), (ins VME_ACC:
    $acc_wb, VR:$vs1), "vme.addh.va", "$acc, $vs1">
    { let Constraints = "$acc = $acc_wb"; }

// --- 2. Data Movement Operations ---
def VME_MOV_ACC_V : RVInstV<0b000001, 0b111,
    OPC_Custom_3, (outs VR:$vd), (ins VME_ACC:$acc,
    GPR:$rs1), "vme.mov.acc.v", "$vd, $acc, $rs1">;

def VME_MOV_V_ACC : RVInstV<0b000010, 0b111,
    OPC_Custom_3, (outs VME_ACC:$acc), (ins VME_ACC:
    $acc_wb, VR:$vs2, GPR:$rs1), "vme.mov.v.acc", "
    $acc, $vs2, $rs1"> { let Constraints = "$acc =
    $acc_wb"; }

// --- 3. Context Management (Load/Store) ---
def VME_LDR : RVInstI<0b001000, 0b111, OPC_Custom_3
    ,(outs VME_ACC:$acc), (ins GPR:$rs1, simm12:
    $imm12),"vme.ldr", "$acc, ${imm12}(${rs1})">;def
     VME_STR : RVInstS<0b001001, 0b111, OPC_Custom_3
    ,(outs), (ins VME_ACC:$acc, GPR:$rs1, simm12:
    $imm12),"vme.str", "$acc, ${imm12}(${rs1})">;
```

Listing 1: Consolidated LLVM TableGen Definitions for VME

Define a new 2D register class `VME_ACC`
⇒ 8 32x32 FP32 MRF

Define new instructions & Map into RISC-V Custom-3 space

## (2) Functional Simulation: Spike (C++)

```
// 1. vme_mopa_vv.h: Matrix Outer-Product
    Accumulation
auto& V1 = P.get_vreg(insn.rs1());
auto& V2 = P.get_vreg(insn.rs2());
size_t elms = P.VU.vlmax;

for (size_t r = 0; r < elms; ++r) {
    float val_row = V2.get_float32(r);
    for (size_t c = 0; c < elms; ++c) {
        float val_col = V1.get_float32(c);
        float* acc_ptr = (float*)&STATE.
    mrf_array[acc_offset + (r * elms + c) * 4];
        *acc_ptr += (val_row * val_col);
    }
}

// 2. vme_mov_acc_v.h: Extract MRF Slice to
    Vector Register
req_t slice_idx = RS1;
auto& VD = P.get_vreg(insn.rd());
for (size_t i = 0; i < elms; ++i) {
    float val = (float)&STATE.mrf_array[
    acc_offset + (slice_idx * elms + i) * 4]; VD
    .set_float32(i, val);
}

// 3. vme_ldr.h: Load Memory to MRF Tile
req_t base_addr = RS1 + insn.i_imm();
for (size_t i = 0; i < elms; ++i) {
    float val = MMU.load_float32(base_addr + i *
     4);
    (float)&STATE.mrf_array[acc_offset + i * 4]
    = val;
}
```

Listing 2: Spike Behavioral Logic for VME ISA

To verify the behavioral logic of the Outer Product Engine (OPE)

- extend the processor state with a `mrf_array`
- apply a ”Golden Model” in the SpikeISA Simulator

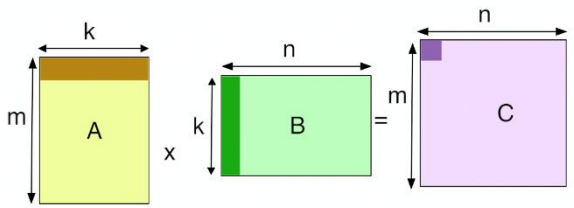
# Comparative Analysis: RVV vs. VME Implementation

```
void gemm_rvv_kernel(float* A, float* B, float* C,
    int M, int N, int K) {
    size_t vl = __riscv_vsetvl_e32m1(N);
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            vfloat32m1_t acc =
    __riscv_vfmv_v_f_f32m1(0.0f, vl);
            for (int k = 0; k < K; k++) {
                vfloat32m1_t va =
    __riscv_vle32_v_f32m1(&A[iK + k], vl);
                float b = B[kN + j];
                // Bottleneck: NK scalar-to-vector
                FMACC callsacc =
    __riscv_vfmacc_vf_f32m1(acc, b, va, vl);
            }__riscv_vse32_v_f32m1(&C[iN + j], acc,
    vl);
        }
    }
}
```

Listing 3: Standard GEMM Kernel using RISC-V Vector (RVV) 1.0

$O(m * n * k)$
scalar-directed loops

(-) has triple-nested loop, issuing FMACC commands for every row-column dot product.




**Refactor a GEMM kernel from using RVV to VME**

(+) ↓ instr issue pressure by VLx

(+) allow VRF & MRF movement ⟶ a hybrid pipeline (2D density +1D flexibility)

```
void gemm_vme_kernel(float* A, float* B, float* C,
    int N) {
    // Stage 1: Fast MRF Reset
    __builtin_riscv_vme_tile_zero(0);

    // Stage 2: Outer-Product Updates (VAU unburdens
     Scalar Core)
    for (int k = 0; k < N; k++) {
        vfloat32m1_t vec_a = __riscv_vle32_v_f32m1(&
    A[k * N], N);
        vfloat32m1_t vec_b = __riscv_vle32_v_f32m1(&
    B[k], N);
        // Atomic MOPA replaces inner loop and
    scalar overhead
        __builtin_riscv_vme_mopa_vv(0, vec_a, vec_b)
    ;
    }

    // Stage 3: Seamless VRF-MRF movement for ReLU
    activation
    for (int i = 0; i < N; i++) {
        vfloat32m1_t res =
    __builtin_riscv_vme_mov_acc_v(0, i);
        res = __riscv_vfmax_vf_f32m1(res, 0.0f, N);
        __riscv_vse32_v_f32m1(&C[i * N], res, N);
    }
}
```

Listing 4: Proposed GEMM Kernel using RISC-V Vector-Matrix Extension (VME)

$O(k)$
matrix-centric atomic operations

# Conclusion

## Evaluation via Architectural Analogy

- No license for environment setup (LLVM, Simulator…)
- Arm SME serve as a reliable proxy

| Metrics | Estimated Improvement |
|---|---|
| Instruction Density | ↓ VL x |
| L1-to-Register Traffic Reduction | 40% – 70% [4] |
| FP32 GEMM Speedup | 3.2x – 3.5x [5] |
| LU Decomposition Speedup | 1.2x – 3.4x [5] |

## Final Remark

- Vector architecture → Matrix Architecture due to massive workload
- Arm SME is a good paradigm for the matrix extension
  - Inherit Vector Length Agnotics & Predicate register from SVE to make Z registers and ZA array scalable.
  - Support versatile ZA access approaches
- Thanks to the RISC-V custom space for extension, we define a few Arm's SME-like VME for RISC-V Vector processor with dedicated Matrix register Files.

## Future challenge

- Area ↑ (due to added Matrix Register Files)
- L2-to-main memory bandwidth saturates. ⟶ Need tighter integration with multi-banked scratchpad memories to sustain the massive data consumption of the OPE.

# Thank you
# for your attention!

# References


[1] Rui Xu, Sheng Ma, Yang Guo, and Dongsheng Li. 2023. A survey of design and Optimization for systolic array-based DNN accelerators. ACM Computing Surveys 56, 1 (June 2023), 1–37. https://doi.org/10.1145/3604802
[2] Leveraging Arm's Scalable Matrix Extension to Accelerate Matrix Multiplication Kernels.pdf
[3] LOw-COst yet High-Performant sparse Matrix-Matrix multiplication on arm SME architectures. Retrieved from https://arxiv.org/html/2511.08158
[4] F. Wilkinson and S. McIntosh-Smith, "An Initial Evaluation of Arm's Scalable Matrix Extension", in 2022 IEEE/ACM International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS), 2022, pp. 135–140, doi: 10 .1109/PMBS56514.2022.00018.
[5] Alhagi Manneh and Karim Haque. 2025. Evaluating Scalable Matrix Extension versus Scalable Vector Extension: Performance Analysis of Numerical Kernels on Modern ARM Architectures. Retrieved from https://kth.diva-portal.org/smash/get/diva2:1985695/FULLTEXT01.pdf
[6] Arm Scalable Vector Extension and application to Machine Learning.pdf
[7] Arm Blog – Part 1: Arm Scalable Matrix Extension (SME) Introduction
[8] Arm Blog - Part 2: Arm Scalable Matrix Extension (SME) Instructions
[9] Arm Blog - Part 3: Matrix-matrix multiplication. Neon, SVE, and SME compared
[10] Arm Website - SME Programmer's Guide
[11] Arm Architecture Reference Manual for A-profile architecture.pdf (w/ 14734 pages…
[12] M. Perotti, S. Riedel, M. Cavalcante, and L. Benini, "Spatz: Clusteringcompact risc-v-based vector units to maximize computing efficiency,"2025. [Online]. Available: https://arxiv.org/abs/2309.10137